

mbqc-cnot-en

1 CNOT gate in MBQC

on a four-qubit T-shaped cluster

1.1 References

- R. Raussendorf and H. J. Briegel, *A One-Way Quantum Computer*, **Physical Review Letters** 86, 5188–5191 (2001), [DOI](#).
- R. Raussendorf, D. E. Browne and H. J. Briegel, *Measurement-based quantum computation on cluster states*, **Physical Review A** 68, 022312 (2003), [DOI](#).

1.2 Environment setup

The following cell installs the dependencies used in this notebook.

You may need to restart the kernel after installing them for the first time.

On Google Colab, setup clones the repository (main branch) and selects the example directory before installing dependencies. If prompted, restart the runtime and rerun the cells. The clone is reused within the same session; start a fresh Colab session to fetch updates from GitHub.

```
[ ]: from pathlib import Path
import os
import subprocess

try:
    import google.colab
except ImportError:
    IN_COLAB = False
else:
    IN_COLAB = True

if IN_COLAB:
    repository = Path("/content/mbqc-codes")
    if not repository.exists():
        subprocess.run([
            "git", "clone", "--depth", "1", "--branch", "main",
            "https://github.com/AlbertoMiserendino/mbqc-codes.git",
            str(repository),
        ], check=True)
```

```
os.chdir(repository / "03-mbqc-cnot")

%pip install -r "../requirements.txt" -q
# If prompted, restart the runtime/kernel and run the cells again.
```

1.3 Minimal four-qubit CNOT pattern

We use the minimal CNOT implementation introduced by Raussendorf and Briegel.

In the notation of the paper, the four physical qubits are arranged as

$$\begin{array}{ccccc} q_1(t_{\text{in}}) & - & q_2 & - & q_3(t_{\text{out}}) \\ & & | & & \\ & & q_4(c) & & \end{array}$$

where:

- qubit 1 contains the input **target** state;
- qubit 4 contains the **control** state;
- qubits 2 and 3 are ancillary qubits initially prepared in $|+\rangle$;
- after the measurements, the target state has been teleported from qubit 1 to qubit 3;
- the control remains on qubit 4 in this minimal implementation.

For computational-basis inputs $|t\rangle_1$ and $|c\rangle_4$,

$$|t\rangle_1 \otimes |+\rangle_2 \otimes |+\rangle_3 \otimes |c\rangle_4$$

is first entangled by the cluster entangling operation S .

Qubits 1 and 2 are then measured in the X basis.

1.4 Measurement outcomes and Pauli byproduct

Let

$$s_1, s_2 \in \{0, 1\}$$

be the outcomes of the X measurements on qubits 1 and 2, respectively, with

$$s_j = 0 \iff |+\rangle_{x,j}, \quad s_j = 1 \iff |-\rangle_{x,j}.$$

The remaining two qubits are left in the state

$$U_{\Sigma}^{(34)}(|t \oplus c\rangle_3 \otimes |c\rangle_4),$$

where

$$U_{\Sigma}^{(34)} = Z_3^{s_1 \oplus 1} X_3^{s_2} Z_4^{s_1}$$

and all exponents are understood modulo 2.

Thus the intended logical operation is a CNOT with qubit 4 as control and the teleported target on qubit 3, followed by a measurement-dependent Pauli byproduct.

1.5 Entangling convention used in the implementation

There is a small convention issue that matters when reproducing the 2001 formula literally.

The original paper defines the cluster entangling operation S through an Ising-type interaction. For this T-shaped four-qubit pattern, its phase convention can be represented in Qiskit as

$$S = Z_3 CZ_{12} CZ_{23} CZ_{24}.$$

The fixed Z_3 is therefore included below so that the simulated byproduct is exactly

$$Z_3^{s_1 \oplus 1} X_3^{s_2} Z_4^{s_1}.$$

If instead one uses the now-common graph-state convention

$$S_{\text{graph}} = CZ_{12} CZ_{23} CZ_{24},$$

the same logical CNOT is obtained, but the fixed extra Z_3 disappears and the byproduct convention becomes

$$Z_3^{s_1} X_3^{s_2} Z_4^{s_1}.$$

1.6 Feed-forward correction

To obtain a deterministic logical CNOT, we remove the Pauli byproduct after the measurements.

Up to an irrelevant global phase,

$$\left(U_{\Sigma}^{(34)}\right)^{-1} = Z_4^{s_1} X_3^{s_2} Z_3^{s_1 \oplus 1}.$$

Therefore the classical feed-forward rules are:

- if $s_1 = 0$, apply Z to the output target q_3 ;
- if $s_2 = 1$, apply X to the output target q_3 ;
- if $s_1 = 1$, apply Z to the control q_4 .

Unlike the arbitrary one-qubit rotation pattern, the measurement bases themselves are not adaptive here: both measurements are simply in the X basis. Only the final Pauli correction depends on the outcomes.

1.7 Dynamic Qiskit circuit

Qiskit uses zero-based indexing, so the paper labels are mapped as

$$1 \mapsto q[0], \quad 2 \mapsto q[1], \quad 3 \mapsto q[2], \quad 4 \mapsto q[3].$$

We prepare generic pure states on the target and control,

$$|\psi_t\rangle = R_z(\phi_t)R_y(\theta_t)|0\rangle, \quad |\psi_c\rangle = R_z(\phi_c)R_y(\theta_c)|0\rangle,$$

rather than restricting the verification to computational-basis states.

This makes the test sensitive to the genuinely quantum action of the CNOT, including the creation of entanglement.

```
[2]: from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit.circuit import Parameter

# Paper labels:
# q[0] = qubit 1 = target input
# q[1] = qubit 2 = measured ancilla
# q[2] = qubit 3 = target output
# q[3] = qubit 4 = control

q = QuantumRegister(4, "q")
s1 = ClassicalRegister(1, "s1")
s2 = ClassicalRegister(1, "s2")

qc = QuantumCircuit(q, s1, s2, name="MBQC_CNOT")

theta_t = Parameter("theta_t")
phi_t = Parameter("phi_t")
theta_c = Parameter("theta_c")
phi_c = Parameter("phi_c")

# Generic target input on paper qubit 1.
qc.ry(theta_t, q[0])
qc.rz(phi_t, q[0])

# Generic control input on paper qubit 4.
qc.ry(theta_c, q[3])
qc.rz(phi_c, q[3])

# Ancillas q2 and q3 are initialized in |+>.
qc.h(q[1])
qc.h(q[2])

qc.barrier()
```

```

# T-shaped cluster edges: (1,2), (2,3), (2,4).
qc.cz(q[0], q[1])
qc.cz(q[1], q[2])
qc.cz(q[1], q[3])

# Fixed phase required to reproduce the entangling convention
# and byproduct formula of Raussendorf-Briegel (2001).
qc.z(q[2])

qc.barrier()

# X measurement of paper qubit 1.
qc.h(q[0])
qc.measure(q[0], s1[0])

# X measurement of paper qubit 2.
qc.h(q[1])
qc.measure(q[1], s2[0])

qc.barrier()

#  $U_{\text{Sigma}^{-1}}$ : measurement-dependent Pauli corrections.
# If  $s1 = 0$ , apply  $Z$  to target output  $q3$ .
with qc.if_test((s1[0], False)):
    qc.z(q[2])

# If  $s2 = 1$ , apply  $X$  to target output  $q3$ .
with qc.if_test((s2[0], True)):
    qc.x(q[2])

# If  $s1 = 1$ , apply  $Z$  to control  $q4$ .
with qc.if_test((s1[0], True)):
    qc.z(q[3])

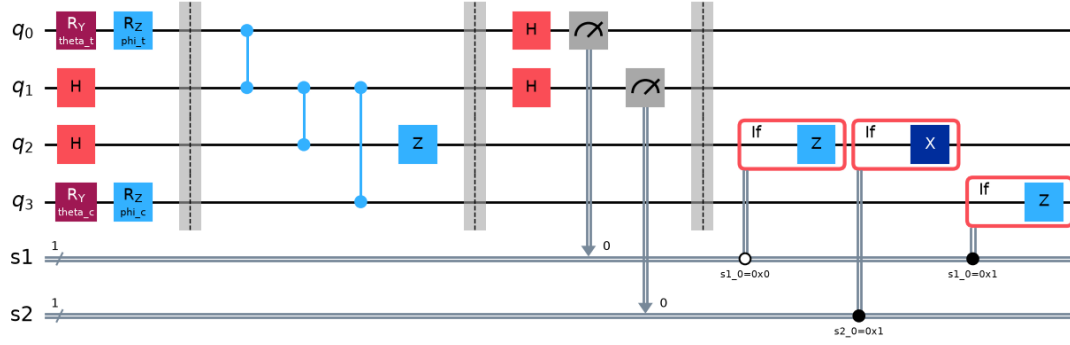
```

```

[3]: # Display the complete dynamic circuit.
qc.draw("mpl", fold=30)

```

[3]:



1.8 Numerical parameters

The circuit has four continuous parameters:

- θ_t, ϕ_t prepare the target input;
- θ_c, ϕ_c prepare the control input.

Generic values are chosen so that both qubits contain nonzero superposition amplitudes and phases. The ideal CNOT output is therefore generally an entangled two-qubit state.

```
[4]: import numpy as np

params = {p.name: p for p in qc.parameters}

theta_t_value = np.pi / 3
phi_t_value = np.pi / 5

theta_c_value = np.pi / 4
phi_c_value = -np.pi / 6

qc_bound = qc.assign_parameters({
    params["theta_t"]: theta_t_value,
    params["phi_t"]: phi_t_value,
    params["theta_c"]: theta_c_value,
    params["phi_c"]: phi_c_value,
})

print("Remaining parameters:", qc_bound.parameters)
```

Remaining parameters: ParameterView([])

1.9 Two-qubit output verification

After the measurements and feed-forward corrections, the logical output qubits are

- $q[2]$: target output t_{out} ;

- $q[3]$: control.

A single Bloch vector is no longer sufficient because the CNOT can entangle the two outputs. We therefore reconstruct the two-qubit density matrix using Pauli tomography.

For two qubits,

$$\rho = \frac{1}{4} \sum_{P, Q \in \{I, X, Y, Z\}} \langle Q \otimes P \rangle (Q \otimes P),$$

where the rightmost Pauli acts on the target and the leftmost Pauli acts on the control in Qiskit's matrix ordering.

Nine measurement settings,

$$(P, Q) \in \{X, Y, Z\}^2,$$

are sufficient to estimate all single-qubit components and all two-qubit correlations.

```
[5]: OUTPUT_TARGET = 2    # paper qubit 3
      OUTPUT_CONTROL = 3  # paper qubit 4

def apply_measurement_basis(circuit, qubit, basis):
    # Rotate X, Y or Z measurement into the computational basis.
    if basis == "X":
        circuit.h(qubit)
    elif basis == "Y":
        circuit.sdg(qubit)
        circuit.h(qubit)
    elif basis != "Z":
        raise ValueError("basis must be 'X', 'Y', or 'Z'")

def circuit_for_two_qubit_measurement(circuit, target_basis, control_basis):
    # Return a copy measuring the two logical outputs in chosen Pauli bases.
    measured = circuit.copy()

    tag = f"{target_basis.lower()}_{control_basis.lower()}"
    out_t = ClassicalRegister(1, f"out_t_{tag}")
    out_c = ClassicalRegister(1, f"out_c_{tag}")
    measured.add_register(out_t)
    measured.add_register(out_c)

    apply_measurement_basis(measured, OUTPUT_TARGET, target_basis)
    apply_measurement_basis(measured, OUTPUT_CONTROL, control_basis)

    measured.measure(OUTPUT_TARGET, out_t[0])
    measured.measure(OUTPUT_CONTROL, out_c[0])
```

```

    idx_t = measured.find_bit(out_t[0]).index
    idx_c = measured.find_bit(out_c[0]).index

    return measured, idx_t, idx_c

measurement_circuits = {}
output_indices = {}

for target_basis in "XYZ":
    for control_basis in "XYZ":
        key = (target_basis, control_basis)
        measured, idx_t, idx_c = circuit_for_two_qubit_measurement(
            qc_bound,
            target_basis,
            control_basis,
        )
        measurement_circuits[key] = measured
        output_indices[key] = (idx_t, idx_c)

```

1.10 Simulation

The circuit contains mid-circuit measurements and classical control, so the nine tomography circuits are transpiled for `AerSimulator` and executed with a finite number of shots.

As in the single-qubit notebook, the earlier measurement outcomes s_1, s_2 remain present in the raw counts because they are needed for feed-forward.

They are marginalized out when reconstructing the logical output state.

```

[6]: from qiskit import transpile
    from qiskit_aer import AerSimulator

    sim = AerSimulator()
    shots = 20_000

    keys = list(measurement_circuits)
    circuits = [measurement_circuits[key] for key in keys]

    transpiled = transpile(circuits, sim)
    result = sim.run(transpiled, shots=shots).result()

    counts = {
        key: result.get_counts(i)
        for i, key in enumerate(keys)
    }

```

```
[7]: # Show a few raw entries for the X-X setting.
from pprint import pprint

pprint(list(counts[("X", "X")].items())[:10])
```

```
[('1 1 0 1', 629),
 ('1 0 1 1', 719),
 ('0 1 1 1', 152),
 ('0 0 1 1', 3481),
 ('0 0 1 0', 3338),
 ('0 0 0 0', 3439),
 ('1 1 0 0', 616),
 ('0 1 0 1', 152),
 ('1 0 0 1', 781),
 ('1 1 1 0', 618)]
```

A raw count key contains the two final output bits together with the earlier measurement registers s_1 and s_2 .

Only the output bits are needed for tomography; the MBQC measurement outcomes have already done their job through the conditional corrections.

1.11 Marginal expectation values

For a single Pauli measurement with outcomes 0 and 1,

$$\langle P \rangle = \frac{N_0 - N_1}{N_0 + N_1}.$$

For a two-qubit Pauli product $Q \otimes P$, the eigenvalue is +1 when the two output bits have even parity and -1 when they have odd parity. Hence

$$\langle Q \otimes P \rangle = \frac{N_{\text{even}} - N_{\text{odd}}}{N_{\text{even}} + N_{\text{odd}}}.$$

```
[8]: from qiskit.result import marginal_counts

def expectation_single(binary_counts):
    # Estimate <P> from one-bit counts.
    n0 = binary_counts.get("0", 0)
    n1 = binary_counts.get("1", 0)
    return (n0 - n1) / (n0 + n1)

def expectation_parity(binary_counts):
    # Estimate a two-qubit Pauli product from output parity.
    even = 0
    odd = 0
```

```

for bitstring, n in binary_counts.items():
    bits = bitstring.replace(" ", "")
    parity = sum(int(bit) for bit in bits) % 2

    if parity == 0:
        even += n
    else:
        odd += n

return (even - odd) / (even + odd)

target_local = {}
control_local = {}
correlations = {}

# Each local expectation is available in three different joint settings.
# Averaging the three estimates reduces sampling noise.
for P in "XYZ":
    estimates = []

    for Q in "XYZ":
        idx_t, _ = output_indices[(P, Q)]
        mt = marginal_counts(counts[(P, Q)], indices=[idx_t])
        estimates.append(expectation_single(mt))

    target_local[P] = float(np.mean(estimates))

for Q in "XYZ":
    estimates = []

    for P in "XYZ":
        _, idx_c = output_indices[(P, Q)]
        mc = marginal_counts(counts[(P, Q)], indices=[idx_c])
        estimates.append(expectation_single(mc))

    control_local[Q] = float(np.mean(estimates))

for P in "XYZ":
    for Q in "XYZ":
        idx_t, idx_c = output_indices[(P, Q)]
        joint = marginal_counts(
            counts[(P, Q)],
            indices=[idx_t, idx_c],
        )

```

```

correlations[(P, Q)] = expectation_parity(joint)

print("Target local expectations:")
for P in "XYZ":
    print(f"<{P}_t> = {target_local[P]: .6f}")

print("\nControl local expectations:")
for Q in "XYZ":
    print(f"<{Q}_c> = {control_local[Q]: .6f}")

```

Target local expectations:

```

<X_t> = 0.692100
<Y_t> = 0.361600
<Z_t> = 0.361067

```

Control local expectations:

```

<X_c> = 0.432200
<Y_c> = -0.241667
<Z_c> = 0.714200

```

1.12 Density-matrix reconstruction

Using the measured Pauli coefficients,

$$\rho_{\text{lin}} = \frac{1}{4} \left[I \otimes I + \sum_P \langle I \otimes P \rangle I \otimes P + \sum_Q \langle Q \otimes I \rangle Q \otimes I + \sum_{P,Q} \langle Q \otimes P \rangle Q \otimes P \right].$$

Finite-shot fluctuations can make a linear-inversion estimate slightly non-positive.

As a small numerical correction, negative eigenvalues are clipped to zero and the matrix is renormalized before computing the fidelity.

```

[9]: from qiskit.quantum_info import Pauli

rho_linear = np.eye(4, dtype=complex)

# Target terms: rightmost Pauli character acts on Qiskit qubit 0
# of the two-qubit logical reference system.
for P in "XYZ":
    rho_linear += target_local[P] * Pauli("I" + P).to_matrix()

# Control terms: leftmost Pauli character.
for Q in "XYZ":
    rho_linear += control_local[Q] * Pauli(Q + "I").to_matrix()

# Two-qubit correlations.
for P in "XYZ":

```

```

    for Q in "XYZ":
        rho_linear += correlations[(P, Q)] * Pauli(Q + P).to_matrix()

rho_linear /= 4

# Enforce Hermiticity against tiny numerical asymmetries.
rho_linear = (rho_linear + rho_linear.conj().T) / 2

eigenvalues, eigenvectors = np.linalg.eigh(rho_linear)

# Project the finite-shot estimate onto the positive semidefinite cone.
eigenvalues_physical = np.clip(eigenvalues, 0, None)
eigenvalues_physical /= eigenvalues_physical.sum()

rho_measured = (
    eigenvectors
    @ np.diag(eigenvalues_physical)
    @ eigenvectors.conj().T
)

print("Linear-inversion eigenvalues:")
print(eigenvalues)

print("\nPhysical reconstructed eigenvalues:")
print(np.linalg.eigvalsh(rho_measured))

```

```

Linear-inversion eigenvalues:
[-0.00613718 -0.00141909  0.0050251   1.00253116]

```

```

Physical reconstructed eigenvalues:
[-6.70966331e-17 -4.46071033e-18  4.98741433e-03  9.95012586e-01]

```

1.13 Theoretical reference state

The reference calculation is independent of the MBQC implementation.

We prepare the same target and control states in an ordinary two-qubit circuit and apply a standard CNOT with the control on logical qubit 1 and the target on logical qubit 0:

$$|\psi_{\text{ideal}}\rangle = \text{CNOT}_{c \rightarrow t}(|\psi_c\rangle \otimes |\psi_t\rangle).$$

The reconstructed MBQC density matrix should agree with this ideal state within the statistical fluctuations caused by the finite number of shots.

```

[10]: from qiskit.quantum_info import Statevector, DensityMatrix

# Logical reference ordering:
# reference qubit 0 = target

```

```

# reference qubit 1 = control
reference = QuantumCircuit(2)

reference.ry(theta_t_value, 0)
reference.rz(phi_t_value, 0)

reference.ry(theta_c_value, 1)
reference.rz(phi_c_value, 1)

reference.cx(1, 0)

psi_theory = Statevector.from_instruction(reference)
rho_theory = DensityMatrix(psi_theory).data

print("Ideal output statevector:")
print(psi_theory.data)

```

```

Ideal output statevector:
[0.79900663-0.04187416j 0.38741529+0.25159043j 0.19107949+0.01001405j
 0.27794681-0.18050077j]

```

1.14 Pauli-coefficient comparison

The tomography can be checked directly at the level of all 15 nontrivial two-qubit Pauli coefficients. For an ideal density matrix ρ_{th} ,

$$\langle A \rangle_{\text{th}} = \text{Tr}(\rho_{\text{th}} A).$$

With $N = 2 \times 10^4$ shots per joint basis, differences of order 10^{-2} are compatible with ordinary finite-sampling fluctuations.

```

[11]: def theoretical_expectation(label):
        operator = Pauli(label).to_matrix()
        return float(np.real(np.trace(rho_theory @ operator)))

rows = []

for P in "XYZ":
    measured = target_local[P]
    theory = theoretical_expectation("I" + P)
    rows.append((f"I{P}", measured, theory, abs(measured - theory)))

for Q in "XYZ":
    measured = control_local[Q]
    theory = theoretical_expectation(Q + "I")
    rows.append((f"{Q}I", measured, theory, abs(measured - theory)))

```

```

for P in "XYZ":
    for Q in "XYZ":
        measured = correlations[(P, Q)]
        theory = theoretical_expectation(Q + P)
        rows.append((f"{Q}-{P}", measured, theory, abs(measured - theory)))

print("Pauli      measured      theoretical      |error|")
for label, measured, theory, error in rows:
    print(f"{label:>4}      {measured: .6f}      {theory: .6f}      {error:.6f}")

print("\nMaximum Pauli-coefficient error:",
      max(error for _, _, _, error in rows))

```

Pauli	measured	theoretical	error
IX	0.692100	0.700629	0.008529
IY	0.361600	0.359943	0.001657
IZ	0.361067	0.353553	0.007513
XI	0.432200	0.429046	0.003154
YI	-0.241667	-0.247710	0.006043
ZI	0.714200	0.707107	0.007093
XX	0.615200	0.612372	0.002828
YX	-0.365000	-0.353553	0.011447
ZX	0.486400	0.495420	0.009020
XY	-0.177200	-0.176777	0.000423
YY	-0.306600	-0.306186	0.000414
ZY	0.515600	0.509037	0.006563
XZ	0.180600	0.179972	0.000628
YZ	0.304600	0.311720	0.007120
ZZ	0.510100	0.500000	0.010100

Maximum Pauli-coefficient error: 0.01144660940672626

1.15 Fidelity

Because the theoretical target is pure, the fidelity can be evaluated directly as

$$F = \langle \psi_{\text{ideal}} | \rho_{\text{meas}} | \psi_{\text{ideal}} \rangle.$$

We also report the purity

$$\text{Tr}(\rho_{\text{meas}}^2),$$

which approaches 1 for a pure reconstructed state.

```
[12]: psi = psi_theory.data
```

```

fidelity = float(np.real(
    np.vdot(psi, rho_measured @ psi)
))

purity = float(np.real(
    np.trace(rho_measured @ rho_measured)
))

delta = rho_measured - rho_theory
trace_distance = 0.5 * float(
    np.sum(np.abs(np.linalg.eigvalsh(delta)))
)

print("Fidelity      =", fidelity)
print("Purity        =", purity)
print("Trace distance =", trace_distance)

```

```

Fidelity      = 0.9949679632005697
Purity        = 0.9900749199497006
Trace distance = 0.008754566339764486

```

1.16 Interpretation

If the cluster entanglement, X measurements and feed-forward corrections are implemented correctly, the two unmeasured qubits contain

$$|\psi_{\text{out}}\rangle = \text{CNOT}_{4 \rightarrow 3} |\psi_{\text{in}}\rangle$$

after the Pauli byproduct has been removed.

The target information has moved from physical qubit 1 to physical qubit 3, while qubit 4 controls whether the logical target is flipped.

For computational-basis inputs this reduces to

$$|t\rangle_1 |c\rangle_4 \mapsto |t \oplus c\rangle_3 |c\rangle_4.$$

For superposition inputs, the same pattern acts coherently and can generate entanglement, which is why the two-qubit tomography above is a stronger verification than checking only the four rows of the classical CNOT truth table.